ESSEC Business School Computer Vision – 2026/2027

Emoji Classification Challenge

Recognising the vendor style of an emoji with a fine-tuned ConvNeXt



One column per vendor, from left to right: apple, facebook, google, messenger, mozilla, samsung, whatsapp.

Task	7-class image classification (emoji vendor)
Model	ConvNeXt-Tiny (CNN), ImageNet-22k pretrained, 5-fold ensemble
Validation	Out-of-fold accuracy **97.56 %**, macro-F1 **0.976**
Leaderboard	Kaggle public score **0.98238**
Deliverables	`emoji_classifier.ipynb`, `submission.csv`, this report

Team

Augustin · Gabriel · Nicolas · Raphaël

October 2026

Abstract

We address the *Emoji Classification Challenge*: given an emoji image, predict which of seven vendors (apple, facebook, google, messenger, mozilla, samsung, whatsapp) drew it. The training set contains 9 879 labelled PNG files and the test set 9 879 unlabelled ones. The difficulty is not *what* is drawn, since every vendor draws the same objects, but *how* it is drawn: line weight, shading, palette, margins and anti-aliasing.

Our solution is a **convolutional neural network**, **ConvNeXt-Tiny** (27.8 M parameters) pretrained on ImageNet-22k and fine-tuned end to end on 160×160 inputs. Most of the work went into the input pipeline. Every PNG colour mode is decoded to *premultiplied* RGBA, padded to a square, resized and composited on white. This removes colour information that is hidden under transparent pixels and differs between vendors. Exact duplicates are found with a pixel fingerprint and kept inside one fold of a *stratified group* 5-fold cross-validation, so the validation score is not inflated. An ablation study showed that the usual geometric data augmentation *lowers* accuracy by 2.7 points, so the final model is trained without augmentation.

The 5-fold ensemble reaches an out-of-fold accuracy of **97.56 %** and a macro-F1 of **0.976**, against 66.7 % for a baseline that only reads file metadata, and its submission scores **0.98238** on the Kaggle public leaderboard. The remaining errors are concentrated on the 4.5 % of images delivered at an unusual (small) resolution: they account for 45 % of the mistakes.

Model chosen at a glance

Family	**CNN**: supervised convolutional network, *not* a Vision-Language Model (VLM) nor a pure Vision Transformer.
Backbone	ConvNeXt-Tiny [1], `timm` weights `convnext_tiny.fb_in22k_ft_in1k`.
Head	Global average pooling → LayerNorm → linear layer 768 → 7 (new, randomly initialised).
Learning	Transfer learning: full fine-tuning, AdamW, one-cycle cosine schedule, label smoothing.
Inference	Average of the 5 fold models' softmax outputs, then an exact-duplicate lookup.

Contents

Abstract **1**

1 Introduction and choice of model **3**
1.1 The task . . . 3
1.2 Why a CNN, and why ConvNeXt? . . . 3
1.3 Overview of the pipeline . . . 4

2 Preprocessing **4**
2.1 Exploring the data . . . 4
2.2 Careful decoding of transparency . . . 5
2.3 Exact duplicates and honest validation folds . . . 6
2.4 Data augmentation: a measured decision to use none . . . 6

3 Architecture of the model **7**
3.1 ConvNeXt-Tiny . . . 7
3.2 Transfer learning and training procedure . . . 8

4 Metrics chosen **9**

5 Performance **9**
5.1 Overall results . . . 9
5.2 Learning curves . . . 10
5.3 Per-class performance and confusion matrix . . . 10
5.4 Where do the errors come from? . . . 11
5.5 Ablation study . . . 12
5.6 Test predictions . . . 13

6 AI and Code **13**
6.1 Code organisation . . . 13
6.2 Reproducibility . . . 14
6.3 Use of AI tools . . . 14

7 Conclusion **15**

1 Introduction and choice of model

1.1 The task

Each image of the challenge is one emoji rendered by one of seven vendors. The label is the vendor, not the emoji: a "frog" drawn by Apple and a "frog" drawn by Google belong to two different classes, while a frog and a car drawn by Google belong to the same class. A classifier therefore has to ignore the semantic content and focus on the *graphic style*. Figure 1 shows random training images per vendor. Some styles are easy to recognise (Mozilla's flat design, Samsung's glossy look), others are close to each other (Facebook and Messenger, two Meta products).



Figure 1: Ten random training emojis per vendor (composited on white). The class is the drawing style, not the object.

The submission is a CSV file with columns `Id,Label` for the 9 879 test images.

1.2 Why a CNN, and why ConvNeXt?

We considered three families of models.

- **Vision-Language Models (CLIP-like, or generative VLMs).** They are trained to match images with captions, so their features describe *what* is in the image ("a green frog") and are, by design, fairly invariant to rendering style. Zero-shot prompts such as "an Apple emoji" would rely on knowledge the model is unlikely to have at the pixel level, and a VLM is also much heavier to run on 9 879 test images. The task does not need language.
- **Vision Transformers.** They are competitive when pretrained at scale, but they work on 16×16 patches and have weaker locality priors. On a dataset of about 10 000 images and at a low input resolution, a CNN is usually easier to fine-tune.
- **Convolutional networks.** Their inductive biases (local filters, weight sharing, translation equivariance) fit what separates vendors: edge sharpness, anti-aliasing, gradients and textures. They are also cheap enough to train a 5-fold ensemble on a laptop GPU.

We chose **ConvNeXt-Tiny** [1], a "modernised" CNN that brings design choices of Vision Transformers (patchify stem, large 7×7 depthwise kernels, LayerNorm, GELU, inverted bottlenecks)

into a pure convolutional network. Its ImageNet-22k pretraining (about 14 M images [2]) gives very good transfer to small datasets. During exploration we also tried 224 px inputs, 15 epochs, ConvNeXt-Small and EfficientNetV2-S on a single fold: none improved on ConvNeXt-Tiny at 160 px by more than the noise level, and they cost 1.5 to 2× more training time.

1.3 Overview of the pipeline

Figure 2 summarises the full pipeline, which the next sections describe step by step.

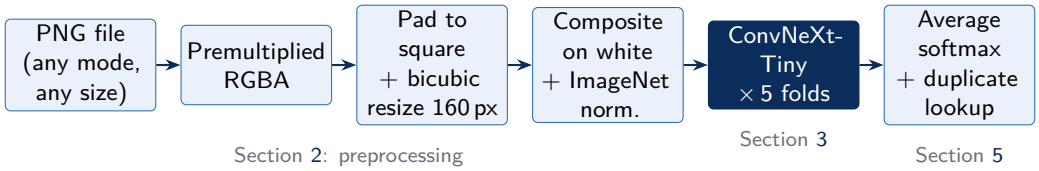


Figure 2: End-to-end pipeline, from a raw PNG file to the predicted vendor.

2 Preprocessing

2.1 Exploring the data

Class balance. The classes are imbalanced (Table 1): mozilla (397 images) and messenger (580) are four to five times rarer than apple (1 924). This is why we report the **macro-F1** next to accuracy (Section 4) and use *stratified* folds.

The files are not uniform. Unlike MNIST-style datasets, the images come in eight native sizes and five PNG colour modes, and both are strongly tied to the vendor (Table 1). For example, almost all whatsapp images are 160 × 160, while facebook, messenger and mozilla are mostly 240 × 240, and apple, google and samsung are mostly 72 × 72. samsung is the only vendor that mostly uses palette (P) images.

Table 1: Training set per vendor: number of images, most frequent native size and PNG colour mode (with the share of the vendor's images they represent).

Vendor	Images	Share (%)	Main native size	Main PNG mode
apple	1 924	19.5	72 × 72 (99.9 %)	RGBA (93.9 %)
facebook	1 667	16.9	240 × 240 (93.0 %)	RGBA (99.0 %)
google	1 877	19.0	72 × 72 (93.6 %)	RGBA (98.0 %)
messenger	580	5.9	240 × 240 (98.6 %)	RGBA (99.5 %)
mozilla	397	4.0	240 × 240 (91.2 %)	RGBA (95.2 %)
samsung	1 790	18.1	72 × 72 (99.7 %)	P (88.3 %)
whatsapp	1 644	16.6	160 × 160 (90.3 %)	RGBA (96.9 %)
Total	9 879	100.0	8 native sizes, 5 PNG modes (RGBA, P, LA, RGB, L)	

A metadata-only baseline. To measure how much these file properties reveal, we trained a decision tree on the one-hot encoded native size and colour mode only, without reading a single pixel. Evaluated with the same folds as the CNN, it reaches **66.7 %** accuracy but only **0.486** macro-F1: it cannot separate the many 72 × 72 RGBA images, nor the 240 × 240 vendors. It is a shortcut, not the visual style we are asked to recognise, so the network below receives *pixels only*: every image is brought to the same square size and the same background. (The original resolution remains indirectly visible, because a 72 px image upscaled to 160 px is blurrier. This cannot be avoided.)

2.2 Careful decoding of transparency

The problem: hidden colours. All files are PNGs with transparency (RGBA, palette with a transparent index, grey-scale with alpha…). A fully transparent pixel is invisible, *but it still stores an RGB value*, and that value depends on the software each vendor used to export its images. Table 2 gives the mean colour stored under fully transparent pixels. The common call `Image.open(f).convert("RGB")` simply drops the alpha channel and exposes these colours to the network (top row of Figure 3).

Table 2: Mean RGB value stored under fully transparent pixels ($\alpha = 0$), over 200 training images per vendor. Samsung hides a grey-blue colour, Mozilla almost pure black.

	apple	facebook	google	messenger	mozilla	samsung	whatsapp
Mean R	42.1	80.8	29.2	36.6	8.9	71.6	64.9
Mean G	36.3	65.7	26.0	29.9	8.9	97.7	52.2
Mean B	28.6	57.1	20.8	24.8	8.5	100.3	42.4

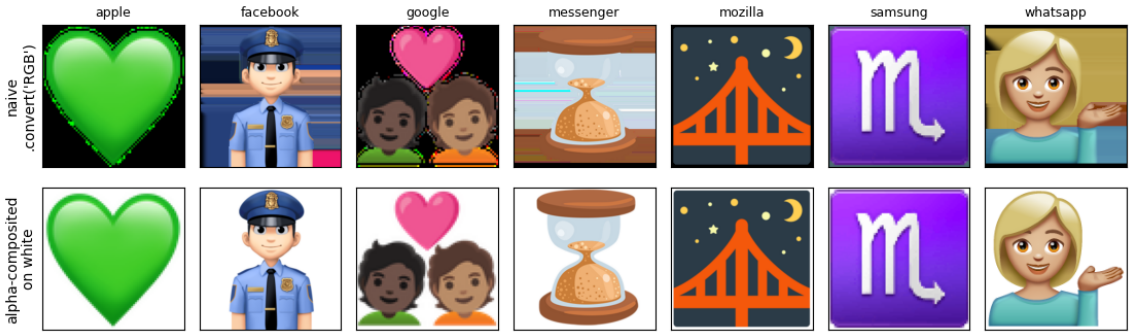


Figure 3: Top: naive conversion to RGB, where the hidden colours under transparent pixels become visible (black, stripes, coloured backgrounds). Bottom: our preprocessing, alpha-composited on white.

These hidden colours are a potential **shortcut**: a network could learn "grey-blue background ⇒ samsung" instead of the drawing style, and such a rule would break as soon as the images are exported by another tool. We remove it with **premultiplied alpha** [3].

Our decoding pipeline. For every file (function `load_premultiplied`):

1. convert any mode (`RGBA`, `P`, `LA`, `L`, `RGB`) to RGBA with values $c \in [0,1]^3$ and $\alpha \in [0,1]$;
2. **premultiply** the colour by the opacity:

$$\tilde{c} = \alpha\, c. \tag{1}$$

   Hidden colours under $\alpha = 0$ become exactly 0, and semi-transparent edge pixels are weighted correctly;
3. **pad** non-square images (e.g. 72×68) with transparent pixels, centred, so that they are not distorted;
4. **resize** to 160×160 with an anti-aliased bicubic filter. Resizing premultiplied colours avoids the dark or coloured halos that appear when straight RGBA is resized. Because bicubic interpolation can overshoot, we clamp $0 \le \tilde{c} \le \alpha$;
5. store the 4-channel result as `uint8` in memory (about 1 GB each for train and test);
6. at batch time, on the GPU, **composite on white**:

$$\text{rgb} = \tilde{c} + (1 - \alpha) \cdot \mathbf{1}, \tag{2}$$

then standardise with the ImageNet statistics used during pretraining, $x = (\text{rgb} - \mu)/\sigma$ with $\mu = (0.485, 0.456, 0.406)$ and $\sigma = (0.229, 0.224, 0.225)$.

We checked that after compositing, every fully transparent pixel has the value 1.0 (pure white) for every vendor. Figure 4 shows what the network actually receives.



Figure 4: Network inputs after preprocessing (160×160, premultiplied, composited on white).

2.3 Exact duplicates and honest validation folds

During decoding we compute a **fingerprint** of each image: an MD5 hash of its native-resolution premultiplied pixels (plus its shape). Two files share a fingerprint exactly when they look identical; hidden colours are ignored. This revealed that:

- the training set holds 9 721 distinct images: **141 duplicate groups** cover 299 files. They are mostly people emojis whose default and gendered versions (e.g. *detective* and *man detective*) are drawn identically by the vendor;
- the copies of an image **always carry the same label** (0 conflicting groups);
- **352 test images** are pixel-identical to at least one training image.

If two copies of the same image fell into different folds, the model would be validated on an image it had already seen, and the validation score would be optimistic. We therefore build the 5 folds with `StratifiedGroupKFold`, using the fingerprint as the group: all copies of an image stay in the same fold, and each fold keeps the class proportions (7 903 training and 1 976 validation images per fold). The code asserts that no fingerprint is shared between the training and validation parts of a fold.

2.4 Data augmentation: a measured decision to use none

Data augmentation is usually essential on a dataset of this size. Here we tested it and decided **not** to use it. The code still contains the augmentations, run on the GPU and switched on with configuration flags, so that the ablation (Section 5.5) can be reproduced:

- **Geometric augmentation** (`geometric_aug`): random rotation of $\pm 10°$, zoom in $[0.85, 1.15]$ and translation of $\pm 8\,\%$, applied with one affine transform per image and bilinear resampling. The border is filled with white by computing $1 - \text{grid_sample}(1 - x)$. Result: -2.7 accuracy points.
- **Horizontal flip** (`hflip`, plus flip test-time augmentation `tta_hflip`): an exact mirror, with no resampling. Result: no measurable effect (97.52 % vs 97.56 % OOF accuracy over the full 5-fold cross-validation).
- **Colour jitter**: excluded by design, because the colour palette is part of a vendor's style.

Why augmentation hurts here

Each vendor draws its emojis on a fixed canvas, with consistent margins, size and position, and renders them with crisp, vendor-specific anti-aliasing. Rotating or zooming resamples the image, which shifts and blurs exactly these cues. Unlike natural photos, the test images come from the same rendering process as the training images, so there is no viewpoint variation for augmentation to cover.

3 Architecture of the model

3.1 ConvNeXt-Tiny

ConvNeXt [1] starts from a ResNet and modernises it step by step with ideas from Vision Transformers. The Tiny variant has four stages with $(3, 3, 9, 3)$ blocks and $(96, 192, 384, 768)$ channels. Figure 5 gives the full network for our 160×160 inputs, with the shape of the feature maps after each stage.

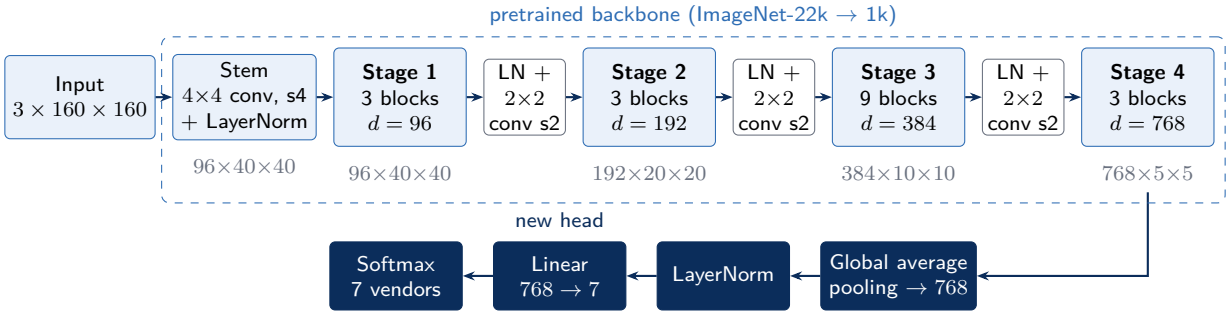


Figure 5: ConvNeXt-Tiny on a 160×160 input. The backbone downsamples by 32 in total, so the last stage works on a 5×5 grid; a new 7-way classification head replaces the 1000-class ImageNet head.

The ConvNeXt block. Every block of a stage has the same structure (Figure 6). With x a d-channel feature map:

$$y = x + \gamma \odot \mathrm{Linear}_{4d \to d}\Big(\,\mathrm{GELU}\,\big(\,\mathrm{Linear}_{d \to 4d}(\mathrm{LN}(\mathrm{DWConv}_{7\times 7}(x)))\big)\Big), \tag{3}$$

where γ is a learned per-channel *layer scale*. The design choices that come from Transformers are:

- a 7×7 **depthwise convolution** mixes information spatially, channel by channel, with a large receptive field (similar to the local attention window of a Swin Transformer);
- an **inverted bottleneck** ($d \to 4d \to d$, implemented as 1×1 convolutions / linear layers) mixes the channels, like the MLP of a Transformer block;
- a single **LayerNorm** and a single **GELU** per block, instead of BatchNorm and ReLU after every convolution;
- a **patchify stem** (4×4 convolution with stride 4) and **separate downsampling layers** between stages.

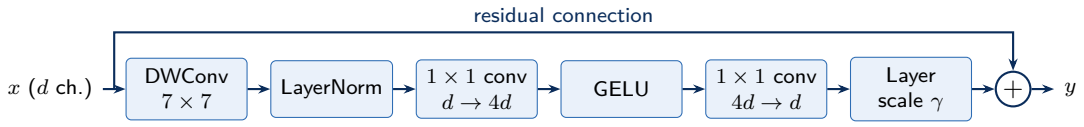


Figure 6: A ConvNeXt block (stochastic depth is disabled in our configuration).

Size of the model. Table 3 breaks down the 27 825 511 parameters. Almost all of them are pretrained: the head has 6 919 parameters, and only the 5 383 of its final linear layer are new. Because the network is fully convolutional and ends with a global pooling, it accepts our 160 px inputs even though it was pretrained at 224 px.

Table 3: Parameters of ConvNeXt-Tiny with a 7-class head (stage counts include the downsampling layer in front of the stage).

Part	Output shape (160 px input)	Parameters	Share (%)
Stem	$96 \times 40 \times 40$	4 896	0.02
Stage 1 (3 blocks)	$96 \times 40 \times 40$	237 888	0.85
Stage 2 (3 blocks)	$192 \times 20 \times 20$	992 256	3.57
Stage 3 (9 blocks)	$384 \times 10 \times 10$	11 112 960	39.94
Stage 4 (3 blocks)	$768 \times 5 \times 5$	15 470 592	55.60
Head (LN + linear)	7	6 919	0.02
Total		27 825 511	100.00

3.2 Transfer learning and training procedure

We use the `timm` [7] weights `convnext_tiny.fb_in22k_ft_in1k` (pretrained on ImageNet-22k, then fine-tuned on ImageNet-1k). `timm` replaces the classifier by a randomly initialised 7-way linear layer, and the *whole* network is then fine-tuned. Table 4 lists the hyper-parameters.

Table 4: Training configuration (the `CFG` dictionary of the notebook).

Setting	Value	Comment
Input size	160×160	At least the native size of every image except the 240×240 ones; cheaper than 224 at no measurable loss
Batch size	64	123 steps per epoch (incomplete last batch dropped)
Epochs	12	1 476 optimisation steps per fold
Optimiser	AdamW [4]	learning rate $3 \cdot 10^{-4}$, weight decay 0.05
LR schedule	One-cycle [5]	linear warm-up from lr/25 over the first 10 % of steps, then cosine decay to ≈ 0; β_1 cycled between 0.95 and 0.85
Loss	Cross-entropy	label smoothing $\varepsilon = 0.1$ [6]
Precision	bfloat16 autocast	mixed precision, `channels_last` memory format
Augmentation	none	see Section 2.4
Seeds	42 + fold	deterministic cuDNN, fixed Python / NumPy / PyTorch seeds

Warm-up. The new head starts from random weights. Its first gradients are large and could damage the pretrained features, so the learning rate starts 25 times lower and only reaches its maximum after 10 % of the steps.

Label smoothing. The target distribution puts $1 - \varepsilon + \varepsilon/K$ on the true class and ε/K on each of the $K - 1 = 6$ others:

$$\mathcal{L} = -\sum_{k=1}^{K} q_k \log p_k, \qquad q_k = (1-\varepsilon)\mathbb{1}[k = y] + \frac{\varepsilon}{K}. \tag{4}$$

This prevents over-confident predictions. It also means that the training loss cannot go below the entropy of q, $H(q) \approx 0.446$; the observed final training loss (0.4465) shows that the network fits the training images almost perfectly.

Cross-validation ensemble. For each of the 5 folds we fine-tune a fresh pretrained model on 4/5 of the data, predict the held-out fifth (the *out-of-fold*, OOF, predictions) and predict the test set. The final test probabilities are the average of the 5 models:

$$\hat{p}(x) = \frac{1}{5}\sum_{f=1}^{5} \operatorname{softmax}\left(g_{\theta_f}(x)\right), \qquad \hat{y}(x) = \arg\max_k \hat{p}_k(x). \tag{5}$$

Duplicate lookup. Finally, when a test image is pixel-identical to a training image (same fingerprint, Section 2.3), we copy the training label. This is a 1-nearest-neighbour rule at distance zero. It is safe because identical training images never carry different labels.

4 Metrics chosen

- **Accuracy**, the share of correctly classified images, is the natural headline metric.
- **Macro-F1** gives every vendor the same weight, whatever its size:

$$\mathrm{F1}_k = \frac{2\,P_k R_k}{P_k + R_k}, \qquad \text{macro-F1} = \frac{1}{K}\sum_{k=1}^{K} \mathrm{F1}_k, \tag{6}$$

  with P_k and R_k the precision and recall of vendor k. With mozilla at only 4 % of the data, a model could ignore it and lose only 4 accuracy points; macro-F1 would expose it. The metadata baseline illustrates the difference: 66.7 % accuracy but 0.486 macro-F1.
- **Per-class precision and recall** and the **confusion matrix** show which vendors are confused with which.
- **Validation log-loss** is only used to monitor training. With label smoothing, the predicted probabilities are capped at about 0.91, so its absolute value is not meaningful.

Evaluation protocol. All the scores below are **out-of-fold**: every training image is predicted by the one model that did not see it during training, and duplicates never cross folds. The OOF score is therefore an estimate of the score on unseen data computed on all 9 879 images, rather than on a single 20 % hold-out.

5 Performance

5.1 Overall results

Table 5: Headline results: out-of-fold on the 9 879 training images, and Kaggle public leaderboard.

Model	Accuracy (%)	Macro-F1
Metadata only (decision tree on native size + PNG mode)	66.70	0.486
ConvNeXt-Tiny, 5-fold ensemble (final)	**97.56**	**0.976**
Kaggle public leaderboard score (part of the test set)	**0.98238**	

Table 6: Score of each fold model on its validation fold (last epoch), and the OOF summary.

	Fold 0	Fold 1	Fold 2	Fold 3	Fold 4	Mean	Std
Accuracy (%)	97.87	96.61	97.47	97.82	98.03	97.56	0.57
Macro-F1 (%)	97.99	96.78	97.40	97.89	98.05	97.62	0.54

Across the full OOF predictions, the model reaches **97.56 %** accuracy, a macro-F1 of **0.9762** and a weighted F1 of 0.9757 (Table 5). Only **241** of the 9 879 images are misclassified. The fold scores are stable (96.6 % to 98.0 %, standard deviation 0.6 point). The correct vendor is among the two most probable ones for 98.98 % of the images (top-2 accuracy).

Kaggle leaderboard. The submitted predictions of the 5-fold ensemble score **0.98238** on the public leaderboard. Kaggle computes this score on part of the test set, with the metric defined on the competition's Evaluation tab; the private leaderboard, computed on the rest of the test set, is not available yet. The public score is in line with the cross-validation estimate, and slightly above it. This is expected: each out-of-fold prediction comes from a single fold model, while each test prediction averages five of them. Moreover, the public part of the test set is only a sample, so its score carries a sampling uncertainty of a few tenths of a point.

5.2 Learning curves

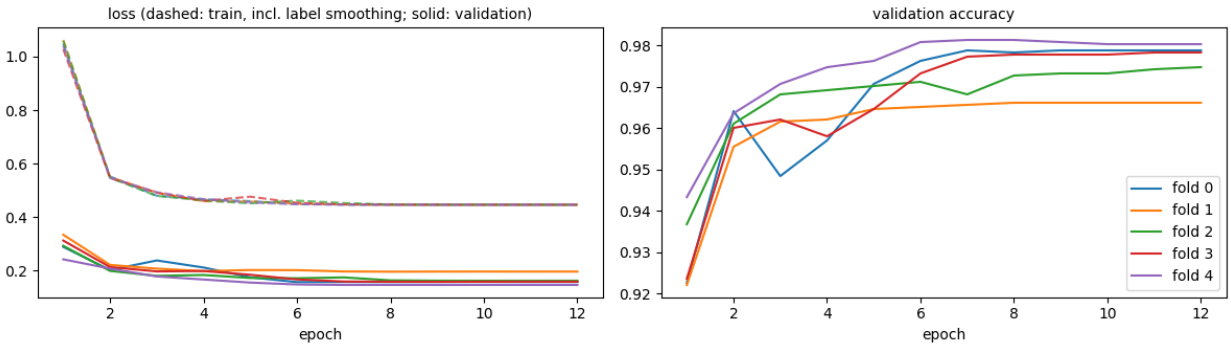


Figure 7: Left: training loss (dashed, includes label smoothing) and validation loss (solid) per fold. Right: validation accuracy per epoch.

Thanks to the pretraining, validation accuracy is above 92 % after the first epoch and above 95 % after the second (Figure 7). It levels off around epoch 7–8, when the cosine schedule has brought the learning rate close to zero. The validation loss never rises, so 12 epochs do not over-fit, even though the training loss reaches its floor of ≈ 0.446. The training time is about 16 s per epoch, i.e. 17.4 minutes for the 5 folds on an NVIDIA RTX 4070 Laptop GPU.

5.3 Per-class performance and confusion matrix

Table 7: Per-vendor out-of-fold precision, recall and F1.

Vendor	Precision	Recall	F1	Images
apple	0.940 0	0.976 6	0.957 9	1 924
facebook	0.988 4	0.968 2	0.978 2	1 667
google	0.973 2	0.986 7	0.979 9	1 877
messenger	0.977 7	0.982 8	0.980 2	580
mozilla	0.979 6	0.969 8	0.974 7	397
samsung	0.988 7	0.980 4	**0.984 6**	1 790
whatsapp	0.993 7	0.962 9	0.978 1	1 644
Macro average	0.977 3	0.975 3	0.976 2	9 879

Table 8: Out-of-fold confusion matrix (rows: true vendor, columns: predicted vendor). Off-diagonal cells with at least 5 errors are shaded. The apple column collects 120 of the 241 errors.

true \ pred.	apple	facebook	google	messenger	mozilla	samsung	whatsapp
apple	**1879**	6	25	0	1	10	3
facebook	33	**1614**	7	6	1	0	6
google	21	1	**1852**	0	1	2	0
messenger	0	4	1	**570**	5	0	0
mozilla	1	1	3	7	**385**	0	0
samsung	28	0	6	0	0	**1755**	1
whatsapp	37	7	9	0	0	8	**1583**

Three patterns stand out (Table 8):

- **"Apple" is the default answer when the model is unsure.** Half of all errors are predicted as apple, which gives it the lowest precision (0.940). Apple is the largest class and, as shown below, its native 72×72 format is shared with many hard images.
- **Facebook vs Messenger** (10 errors in the two directions): both are Meta products and share part of their design language.
- **Messenger vs Mozilla** (12 errors in the two directions): two flat, 240×240 styles, and the two smallest classes.

5.4 Where do the errors come from?

Most vendors deliver almost all their emojis at one "usual" size (Table 1), but a few hundred images come at another, usually smaller, size. Table 9 compares the two groups.

Table 9: OOF accuracy by native size. Top: images at their vendor's usual size vs. the others. Bottom: detail of the vendor/size combinations with at least 5 images.

Group		Images	Accuracy (%)	Errors
Vendor's usual size		9430	98.6	133
Other size		449	**76.0**	108
Vendor	**Native size**			
facebook	240×240 (usual)	1550	99.2	
	72×72	117	65.0	
whatsapp	160×160 (usual)	1484	99.7	
	72×72	147	65.3	
	64×64	13	53.8	
mozilla	240×240 (usual)	362	97.8	
	72×72	35	88.6	
samsung	72×72 (usual)	1784	98.2	
	108×108	6	50.0	
google	72×72 (usual)	1756	98.6	
	72×68	116	100.0	

The model is very accurate on images at their vendor's usual size (98.6 %), but much weaker on the "off-size" ones (76.0 %). They are **4.5 % of the data but 45 % of the errors**. At 72 px much of the fine detail that separates vendors is gone. Moreover, 72×72 is precisely the usual size of apple, google and samsung, so a small Facebook or WhatsApp emoji is mostly predicted as one of those three, which explains the apple column of the confusion matrix. Only a few dozen to about 150 training examples per vendor exist at those sizes. Google's 72×68 images are the exception: they are almost the same size as the usual 72×72 and are classified perfectly.



Figure 8: The 24 most confident out-of-fold mistakes (true vendor, predicted vendor and probability). Many are blurry upscaled small images, or simple symbols (check mark, cross, snowflake) that carry little vendor-specific style.

5.5 Ablation study

To check that each design choice is useful, we changed **one** element of the final recipe at a time and retrained on fold 0 with the same seed, epochs and optimiser (Table 10). A fold has 1 976 validation images, so one image is worth 0.05 accuracy points, and differences of a few tenths of a point (5 to 8 images) are within the noise between splits and seeds.

Table 10: Ablation on fold 0 (1 976 validation images).

Variant	Accuracy (%)	Δ (pts)	Macro-F1
Final recipe (premultiplied alpha, no augmentation)	**97.87**	–	**0.9799**
+ geometric augmentation (rotation, zoom, translation)	95.14	−2.73	0.9537
+ horizontal flip (training + test-time)	97.47	−0.40	0.9754
Naive `convert("RGB")` instead of premultiplied alpha	97.82	−0.05	0.9776

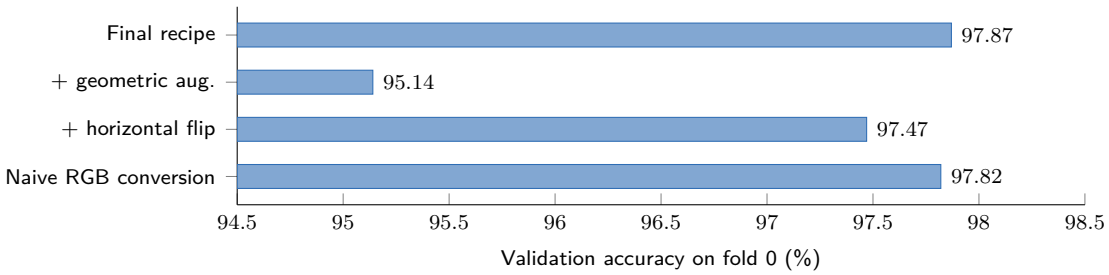


Figure 9: Ablation results. Only geometric augmentation has an effect larger than the noise, and it is negative.

- **Geometric augmentation:** −2.7 **points**, a clear loss (it was also the largest effect in our preliminary experiments).
- **Horizontal flip (+ flip TTA):** −0.4 **points** here, but +0.4 points in a preliminary experiment on another split. Over the full 5-fold cross-validation the two recipes are equivalent (97.52 % vs 97.56 % OOF accuracy), so we keep the simpler pipeline.
- **Naive RGB conversion:** −0.05 **points**, i.e. the same score. The network does not need the

hidden colours to reach this accuracy. The clean preprocessing costs nothing and guarantees that the model only uses what is actually visible.

5.6 Test predictions

The 5 fold models are averaged on the test set. Of the 352 test images that are identical to a training image, the ensemble already predicted the correct training label for all 352, so the duplicate lookup changes **0** predictions. It acts as a safety net and as an independent check of the model. The predicted class distribution on the test set is very close to the training distribution (Table 11), which suggests the test set is drawn from the same distribution. Only 64 test images have a maximum predicted probability below 0.5.

Table 11: Class shares in the training labels and in our test predictions (`submission.csv`).

	apple	facebook	google	messenger	mozilla	samsung	whatsapp
Training share (%)	19.5	16.9	19.0	5.9	4.0	18.1	16.6
Test predicted share (%)	19.6	15.9	19.6	6.1	4.2	18.3	16.3

6 AI and Code

6.1 Code organisation

The whole solution is a single Jupyter notebook, `emoji_classifier.ipynb`, that runs top to bottom and writes every result to disk. Table 12 maps its sections to this report.

Table 12: Structure of the notebook and of the repository.

Notebook section	Content and main functions	Report
0. Setup	package versions, `CFG` dictionary, `seed_everything`	§3
1. The data	class balance, native sizes and PNG modes, sample grid	§2
2. Preprocessing	`load_premultiplied`, `load_folder`, `composite_on_white`; fingerprints, duplicates, `StratifiedGroupKFold` folds; near-duplicate check with perceptual hashes (added later, not executed yet)	§2
3. Baseline	metadata-only decision tree	§2
4. Model and training	`normalize`, `augment`, `predict_proba`, `train_one_fold`; 5-fold loop	§3
5. Evaluation	OOF metrics, learning curves, confusion matrix, error analysis	§5
6. Submission	fold averaging, duplicate lookup, format checks, `submission.csv`	§5
7. Ablation	one-change-at-a-time variants on fold 0	§5.5
8. Experiments	oversampling of the off-size images, nearest-neighbour upscaling, 8 epochs, final model retrained on 100 % of the data with 3 seeds (added later, not executed yet)	§7
9. Summary	results and lessons	§7

Besides the notebook, the repository contains `requirements.txt` (pinned package versions), the training history and ablation results (`outputs/history.csv`, `outputs/ablation.csv`), the out-of-fold and test probabilities (`outputs/oof_proba.npy`, `outputs/test_proba.npy`), the final `submission.csv`, a `README.md` with setup and running instructions, and `AI_USAGE.md`, the declaration of AI assistance (Section 6.3).

Implementation choices.

- **One configuration dictionary.** Every helper takes a `cfg` argument, so the same code trains the final model and each ablation variant (e.g. `dict(CFG, geometric_aug=True)`).

- **In-memory, GPU-side pipeline.** The preprocessed images are kept as `uint8` tensors (about 1 GB per split), and compositing, normalisation and augmentation run on the GPU per batch. There is no disk cache and no `DataLoader` worker, which also avoids multiprocessing problems with Jupyter on Windows.
- **Built-in checks.** The code asserts that no fingerprint is shared between the training and validation parts of a fold, that duplicates never disagree on their label, and that the submission has one row per test file, in order, with valid labels only.
- **Hardware independence.** Mixed precision uses bfloat16 on recent GPUs and falls back to float16 with gradient scaling otherwise (e.g. a Kaggle T4), or to the CPU.

The core of the training loop is short:

```
for k in range(steps_per_epoch):
    idx = perm[k * bs:(k + 1) * bs]
    img = augment(composite_on_white(X[idx]), cfg) # uint8 premultiplied RGBA -> RGB on white
    target = y_t[idx].to(DEVICE)
    with torch.autocast(DEVICE.type, dtype=AMP_DTYPE, enabled=USE_AMP):
        loss = F.cross_entropy(model(normalize(img, model)), target,
                               label_smoothing=cfg["label_smoothing"])
    opt.zero_grad(set_to_none=True)
    scaler.scale(loss).backward()
    scaler.step(opt); scaler.update(); sched.step()
```

6.2 Reproducibility

All random seeds (Python, NumPy, PyTorch CPU and GPU) are fixed, cuDNN is forced to deterministic algorithms, and `CUBLAS_WORKSPACE_CONFIG` is set before CUDA is initialised. Two complete, independent runs of the notebook gave identical losses and accuracies at every epoch. The exact package versions are pinned in `requirements.txt` (Python 3.12, PyTorch 2.11 with CUDA 12.8, `timm` 1.0.30, scikit-learn 1.9.1), and the pretrained weights are downloaded from a fixed Hugging Face Hub commit, so the starting point of the fine-tuning cannot change. On an RTX 4070 Laptop GPU the full run takes about 2 minutes of image loading, 18 minutes of 5-fold training and 11 minutes of ablation.

6.3 Use of AI tools

We used an AI coding assistant, **Claude Code** (Anthropic) with the **Claude Opus 5.5** model. Starting from the competition rules, it designed, wrote and ran the notebook on our GPU, including the exploratory hyperparameter sweep and the ablation study, and it drafted this report from the notebook's outputs. The repository records this: the commits that add the notebook and the report list Claude as a co-author, and `AI_USAGE.md` declares the tools used, the AI-assisted files and notebook cells, how each of them was checked, and every prompt sent to the assistant.

We kept the following rules so that the results can be trusted:

- **Every claim is backed by an executed cell.** All the numbers in this report come from the outputs stored in the notebook and in `outputs/`, not from the assistant's expectations.
- **Defaults are tested, not assumed.** The "standard recipe" for small image datasets includes data augmentation; our ablation showed that, here, it costs 2.7 points. Likewise, the premultiplied-alpha preprocessing was kept because it removes a shortcut, and the ablation confirmed that it costs nothing.
- **Validation is protected against leaks.** Duplicate-aware folds and assertions make sure the reported scores are not inflated, whoever wrote the code.
- **Understanding before submitting.** Each step of the pipeline is explained in the notebook and in this report, and we can justify every hyper-parameter in Table 4.

7 Conclusion

We built a vendor classifier for emojis that reaches **97.56 %** out-of-fold accuracy and a macro-F1 of **0.976**, with a ConvNeXt-Tiny CNN fine-tuned from ImageNet-22k weights and averaged over 5 folds; its submission scores **0.98238** on the Kaggle public leaderboard. Four lessons stand out:

1. **Careful decoding matters more than model size.** Converting every PNG mode to premultiplied RGBA, padding, resizing and compositing on white gives clean and comparable inputs, and removes the hidden-colour shortcut at no cost in accuracy.
2. **Validation must be honest.** Exact duplicates (141 groups) are kept inside one fold, so the OOF score is a reliable estimate; the 352 test images identical to a training image were all already predicted correctly.
3. **Augmentation is not always helpful.** When the class is a rendering style, geometric transformations destroy the signal (-2.7 points).
4. **The remaining errors have a clear cause.** Images delivered at an unusual small size are 4.5 % of the data but 45 % of the errors, mostly predicted as one of the 72×72 vendors.

Limitations and next steps. The main limitation is the low resolution of the "off-size" images. Section 8 of the notebook, added after the results of this report, tests two remedies: over-sampling the off-size images during training and upscaling small images with nearest-neighbour interpolation instead of bicubic. It also tests a shorter 8-epoch schedule and retrains the best setup on 100 % of the training data with 3 seeds; it has not been run yet. Other directions are to give the network the native resolution as an extra input (or a dedicated branch for small images), to ensemble different backbones (e.g. a ConvNeXt with an EfficientNet), and to pseudo-label the most confident test predictions.

References

[1] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, S. Xie. A ConvNet for the 2020s. *CVPR*, 2022.

[2] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, L. Fei-Fei. ImageNet: A large-scale hierarchical image database. *CVPR*, 2009.

[3] T. Porter, T. Duff. Compositing digital images. *SIGGRAPH*, 1984.

[4] I. Loshchilov, F. Hutter. Decoupled weight decay regularization. *ICLR*, 2019.

[5] L. N. Smith, N. Topin. Super-convergence: very fast training of neural networks using large learning rates. *Artificial Intelligence and Machine Learning for Multi-Domain Operations Applications*, SPIE, 2019.

[6] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, Z. Wojna. Rethinking the Inception architecture for computer vision. *CVPR*, 2016.

[7] R. Wightman. PyTorch Image Models (`timm`). https://github.com/huggingface/pytorch-image-models, 2019.